
Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Interpolação e Ajuste de Curvas: Uma Abordagem Integrada em Python

Disciplina: Cálculo Numérico
Professora: Angela Leite Moreno
Modalidade: Atividade de investigação em dupla
Entrega: Relatório .pdf + código-fonte .py

Apresentação

Nos capítulos anteriores, estudamos dois grandes paradigmas para lidar com dados numéricos: a **interpolação**, que constrói uma curva que passa exatamente por todos os pontos fornecidos, e o **ajuste de curvas**, que busca um modelo que melhor representa os dados no sentido dos mínimos quadrados, sem forçar a passagem pelos pontos.

Ambos respondem à mesma pergunta fundamental: “*Dado um conjunto de dados (x_i, y_i) , como representar analiticamente o comportamento subjacente?*” Mas as respostas diferem profundamente em premissas, garantias e riscos.

Esta investigação propõe que você explore, compare e integre os dois métodos a partir de situações concretas — incluindo dados reais, dados com ruído e decisões de engenharia que exigem escolha consciente entre interpolação e ajuste.

Objetivos

Ao concluir esta investigação, o aluno será capaz de:

- a) Implementar e aplicar os métodos de Lagrange, Newton (diferenças divididas) e *spline* cúbica;
- b) Implementar regressão linear, polinomial e linearização de modelos não lineares por mínimos quadrados;
- c) Identificar quando a interpolação é preferível ao ajuste, e vice-versa;
- d) Diagnosticar o fenômeno de Runge e propor mitigações (nós de Chebyshev, *splines*);
- e) Reconhecer o impacto do ruído e da quantidade de dados na escolha do método;
- f) Integrar os dois paradigmas em um projeto completo de análise de dados.

Atenção

O objetivo central desta investigação **não é apenas obter respostas corretas**, mas desenvolver *juízo numérico*: a capacidade de reconhecer o tipo de problema, escolher o método adequado e interpretar criticamente os resultados. Documente todas as observações — inclusive as inesperadas.

Sumário

1	Interpolação: Implementação e Verificação	3
1.1	Verificação básica e equivalência Lagrange–Newton	4
1.2	Fenômeno de Runge e nós de Chebyshev	5
1.3	<i>Spline</i> cúbica vs. polinômio global	6
2	Ajuste de Curvas: Mínimos Quadrados e Modelos	6
2.1	Regressão linear e significado dos mínimos quadrados	8
2.2	Escolha de modelo e sobreajuste	8
3	Dados com Ruído: Interpolação ou Ajuste?	9
4	Modelos Não Lineares e Linearização	10
5	Comparação Integrada: Interpolação vs. Ajuste	12
6	Projeto Integrador: Calibração de Sensor	13
7	Desafio (Opcional — Pontuação Extra)	14

1. Interpolação: Implementação e Verificação

Ponto de partida

Copie os blocos abaixo para o arquivo `interpolacao.py`. Não os modifique antes de responder às questões desta seção.

Listing 1: `interpolacao.py` — Lagrange, Newton e *Spline*

```

1 import numpy as np
2 from scipy.interpolate import CubicSpline
3
4 def lagrange(x_pts, y_pts, x_eval):
5     """
6     Interpolacao de Lagrange.
7     Retorna o valor interpolado em x_eval (escalar ou array).
8     """
9     x_pts = np.array(x_pts, dtype=float)
10    y_pts = np.array(y_pts, dtype=float)
11    n = len(x_pts)
12    x_eval = np.atleast_1d(np.array(x_eval, dtype=float))
13    result = np.zeros_like(x_eval)
14    for i in range(n):
15        L = np.ones_like(x_eval)
16        for j in range(n):
17            if j != i:
18                L *= (x_eval - x_pts[j]) / (x_pts[i] - x_pts[j])
19        result += y_pts[i] * L
20    return result if result.size > 1 else result[0]
21
22
23 def diferencas_divididas(x_pts, y_pts):
24     """
25     Calcula a tabela de diferencas divididas de Newton.
26     Retorna os coeficientes [f[x0], f[x0,x1], f[x0,x1,x2], ...].
27     """
28     x_pts = np.array(x_pts, dtype=float)
29     y_pts = np.array(y_pts, dtype=float)
30     n = len(x_pts)
31     tabela = y_pts.copy()
32     coefs = [tabela[0]]
33     for k in range(1, n):
34         tabela = (tabela[1:] - tabela[:-1]) / (x_pts[k:] - x_pts[:-k])
35         coefs.append(tabela[0])
36     return np.array(coefs)
37
38
39 def newton(x_pts, y_pts, x_eval):
40     """
41     Interpolacao de Newton por diferencas divididas.

```

```

42     Retorna o valor interpolado em x_eval.
43     """
44     x_pts = np.array(x_pts, dtype=float)
45     coefs = diferencas_divididas(x_pts, y_pts)
46     x_eval = np.atleast_1d(np.array(x_eval, dtype=float))
47     n = len(coefs)
48     result = coefs[-1] * np.ones_like(x_eval)
49     for k in range(n - 2, -1, -1):
50         result = result * (x_eval - x_pts[k]) + coefs[k]
51     return result if result.size > 1 else result[0]
52
53
54 def nos_chebyshev(a, b, n):
55     """
56     Retorna n nos de Chebyshev no intervalo [a, b].
57     """
58     k = np.arange(1, n + 1)
59     return 0.5 * (a + b) + 0.5 * (b - a) * np.cos((2*k - 1) * np.pi
60         / (2*n))
61
62 def spline_cubica(x_pts, y_pts, x_eval, tipo='not-a-knot'):
63     """
64     Spline cubica via scipy.interpolate.CubicSpline.
65     tipo: 'not-a-knot', 'natural', 'clamped'
66     """
67     cs = CubicSpline(x_pts, y_pts, bc_type=tipo if tipo != 'not-a-
68         knot' else 'not-a-knot')
69     return cs(x_eval), cs

```

1.1. Verificação básica e equivalência Lagrange–Newton

Q1.1 — Primeiros passos com os métodos

Considere os pontos tabelados:

x	1,0	2,0	3,0	4,0
$f(x)$	1,0	4,0	9,0	16,0

- Execute `lagrange` e `newton` para $x = 1,5$ e $x = 2,5$. Os dois métodos produzem o mesmo resultado? Qual é a diferença numérica $|p_L(x) - p_N(x)|$? Explique por que, teoricamente, os dois devem coincidir.
- Imprima a tabela de diferenças divididas usando `diferencas_divididas`. Identifique os coeficientes do polinômio de Newton na forma:

$$p_3(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + c_3(x - x_0)(x - x_1)(x - x_2).$$

- Os dados acima provêm de $f(x) = x^2$. O polinômio interpolador coincide exatamente com f ? Por quê? (Discuta o teorema de unicidade.)

Q1.2 — Adição de um ponto e atualização de Newton

Adicione o ponto $x_4 = 5, 0$, $f(x_4) = 25, 0$ à tabela anterior.

- Para a forma de Lagrange, quantas operações a mais são necessárias? Refaça o cálculo completo para o novo conjunto.
- Para a forma de Newton, mostre como *apenas uma nova diferença dividida* precisa ser calculada para atualizar o polinômio. Verifique que o novo polinômio coincide com o de Lagrange em $x = 3, 5$.
- Esta propriedade é a principal vantagem prática de Newton sobre Lagrange. Em que cenário ela seria crítica em uma aplicação real?

1.2. Fenômeno de Runge e nós de Chebyshev**Q1.3 — Runge e remédios**

Considere a função de Runge $f(x) = \frac{1}{1 + 25x^2}$, $x \in [-1, 1]$.

- Para $n \in \{5, 9, 13, 17\}$, interpole f com nós **igualmente espaçados** usando Lagrange. Plote o erro $|f(x) - p_n(x)|$ em $[-1, 1]$ (utilize 500 pontos de avaliação). O erro aumenta ou diminui com n ?
- Repita com **nós de Chebyshev** (função `nos_chebyshev`). Plote as duas curvas de erro ($n = 13$) no mesmo gráfico. Qual redução máxima de erro você observa?
- Preencha a tabela:

n	$\ f - p_n\ _\infty$ (espaçados)	$\ f - p_n\ _\infty$ (Chebyshev)
5		
9		
13		
17		

- Por que os nós de Chebyshev mitigam o fenômeno de Runge? Relacione com a fórmula do erro de interpolação:

$$|f(x) - p_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega_{n+1}(x)|, \quad \omega_{n+1}(x) = \prod_{i=0}^n (x - x_i).$$

1.3. *Spline* cúbica vs. polinômio global

Q1.4 — Quando a *spline* supera o polinômio global

Use os dados mensais de temperatura média em Alfenas-MG (dados fictícios, mas plausíveis):

Mês	1	2	3	4	5	6	7	8	9	10	11	12
T (°C)	23	24	23	21	18	16	16	18	20	22	23	24

- Interpole com Lagrange ($n = 11$, nós nos meses $1, 2, \dots, 12$). Plote o polinômio interpolador em $[1, 12]$ com 500 pontos. Há oscilações indesejadas? Onde?
- Interpole com *spline* cúbica (`tipo='not-a-knot'`). Plote na mesma figura. Qual representação é mais fiel ao comportamento físico esperado de uma série de temperaturas?
- Compare os valores interpolados para $x = 1,5$, $x = 6,5$ e $x = 11,5$. A diferença é significativa? Qual método você usaria em um sistema de monitoramento climático? Justifique.
- Investigue o efeito das condições de contorno da *spline*. Execute com `tipo='natural'` e `tipo='not-a-knot'`. Os resultados diferem nos extremos? Por quê?

2. Ajuste de Curvas: Mínimos Quadrados e Modelos

Ponto de partida

Copie o bloco abaixo para `ajuste.py`.

Listing 2: `ajuste.py` — Mínimos quadrados e linearizações

```

1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 def regressao_linear(x, y):
5     """
6     Ajuste linear  $y = a_0 + a_1x$  pelo método dos mínimos quadrados.
7     Retorna  $(a_0, a_1, r\_quadrado)$ .
8     """
9     n = len(x)
10    Sx = np.sum(x); Sy = np.sum(y)
11    Sxx = np.sum(x**2); Sxy = np.sum(x * y)
12    D = n * Sxx - Sx**2
13    a1 = (n * Sxy - Sx * Sy) / D
14    a0 = (Sy - a1 * Sx) / n

```

```

15     y_hat = a0 + a1 * x
16     ss_res = np.sum((y - y_hat)**2)
17     ss_tot = np.sum((y - np.mean(y))**2)
18     r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 1.0
19     return a0, a1, r2
20
21
22 def regressao_polynomial(x, y, grau):
23     """
24     Ajuste polinomial de grau 'grau' por minimos quadrados.
25     Retorna coeficientes (grau crescente) e r_quadrado.
26     """
27     coefs = np.polyfit(x, y, grau)[::-1]    # grau crescente
28     y_hat = np.polyval(coefs[::-1], x)
29     ss_res = np.sum((y - y_hat)**2)
30     ss_tot = np.sum((y - np.mean(y))**2)
31     r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 1.0
32     return coefs, r2
33
34
35 def ajuste_exponencial(x, y):
36     """
37     Lineariza  $y = a * \exp(b*x)$  via  $\ln(y) = \ln(a) + b*x$ .
38     Retorna (a, b, r_quadrado_log).
39     """
40     ln_y = np.log(y)
41     a0, a1, r2 = regressao_linear(x, ln_y)
42     return np.exp(a0), a1, r2
43
44
45 def ajuste_potencia(x, y):
46     """
47     Lineariza  $y = a * x^b$  via log-log:  $\ln(y) = \ln(a) + b*\ln(x)$ .
48     Retorna (a, b, r_quadrado_log).
49     """
50     ln_x = np.log(x)
51     ln_y = np.log(y)
52     a0, b, r2 = regressao_linear(ln_x, ln_y)
53     return np.exp(a0), b, r2
54
55
56 def ajuste_nao_linear(func, x, y, p0=None):
57     """
58     Ajuste nao-linear generico via scipy.optimize.curve_fit.
59     Retorna (params_otimos, cov_matrix).
60     """
61     popt, pcov = curve_fit(func, x, y, p0=p0, maxfev=10000)
62     return popt, pcov
63
64
65 def residuos(x, y, y_hat):

```



```

66     """
67     Retorna vetor de residuos e estatisticas basicas.
68     """
69     res = y - y_hat
70     return res, np.std(res), np.max(np.abs(res))

```

2.1. Regressão linear e significado dos mínimos quadrados

Q2.1 — A geometria dos mínimos quadrados

Considere os dados de velocidade e distância de frenagem de um veículo:

v (km/h)	20	40	60	80	100	120
d (m)	5	14	28	47	72	103

- Ajuste um modelo linear $d = a_0 + a_1 v$ usando `regressao_linear`. Reporte a_0 , a_1 e R^2 . O ajuste é bom?
- Agora ajuste $d = a_1 v^2$ (modelo físico correto para frenagem). Para isso, linearize fazendo $z = v^2$ e ajuste $d = a'_0 + a'_1 z$. Compare o R^2 dos dois modelos. Qual é mais adequado?
- Plote os dados, a reta linear e a parábola no mesmo gráfico. Plote também os resíduos $r_i = d_i - \hat{d}_i$ para cada modelo. O que o gráfico de resíduos revela sobre a adequação do modelo linear?
- A solução de mínimos quadrados minimiza $\sum r_i^2$, não $\sum |r_i|$. Por que essa escolha matemática? Quais são as implicações quando há *outliers*?

2.2. Escolha de modelo e sobreajuste

Q2.2 — Sobreajuste polinomial

Gere 15 dados sintéticos com ruído:

```

np.random.seed(7)
x = np.linspace(0, 2*np.pi, 15)
y = np.sin(x) + 0.15 * np.random.randn(15)

```

- Ajuste polinômios de grau $k \in \{1, 3, 5, 7, 10, 14\}$ usando `regressao_polinomial`. Para cada grau, calcule:
 - R^2 no conjunto de treinamento (os 15 pontos);
 - o erro máximo $\|f - p_k\|_\infty$ avaliado em 500 pontos de $[0, 2\pi]$ em relação à função verdadeira $\sin(x)$.
- Preencha a tabela:

Grau k	R^2 (treino)	$\ f - p_k\ _\infty$
1		
3		
5		
7		
10		
14		

- (c) Plote, em escala log, o erro em função do grau. A partir de qual grau o modelo começa a se deteriorar fora dos pontos de ajuste?
- (d) Este fenômeno chama-se **sobreaajuste** (*overfitting*). Explique a diferença entre *erro de treinamento* e *erro de generalização*. Como isso se relaciona com o conceito de fenômeno de Runge estudado na seção anterior?

Q2.3 — Critérios para escolha do grau

Usando os dados sintéticos da questão anterior:

- (a) Separe aleatoriamente os 15 pontos em 10 de treino e 5 de teste (`np.random.seed(42)`). Ajuste polinômios de grau 1 a 10 nos dados de treino e calcule o erro quadrático médio (MSE) nos dados de teste. Plote MSE-treino e MSE-teste em função do grau. Qual grau minimiza o MSE de teste?
- (b) O critério de informação de Akaike (AIC) penaliza a complexidade do modelo:

$$\text{AIC} = n \ln \left(\frac{\text{RSS}}{n} \right) + 2(k + 1),$$

em que RSS é a soma dos resíduos ao quadrado e k é o grau do polinômio. Calcule o AIC para graus 1 a 10. Qual grau minimiza o AIC?

- (c) Os dois critérios (validação cruzada e AIC) concordam? Discuta qual é mais adequado quando os dados são escassos.

3. Dados com Ruído: Interpolação ou Ajuste?

Questão central desta seção

Quando os dados contêm erros de medição, **interpolar é quase sempre errado**: o método força o polinômio a passar pelos erros. O ajuste por mínimos quadrados *suaviza* o ruído, mas exige que você escolha a família de modelos. Esta seção coloca essa tensão em evidência.

Q3.1 — O custo de interpolar dados ruidosos

Considere medições (com ruído) de uma função exponencial decrescente:

```
np.random.seed(3)
x_med = np.array([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
y_med = 5.0 * np.exp(-0.8 * x_med) + 0.2 * np.random.randn(7)
```

- Aplique a interpolação de Lagrange (grau 6) aos 7 pontos ruidosos. Plote a curva interpoladora, os dados e a função verdadeira $f(x) = 5e^{-0,8x}$ em $[0, 3]$. O interpolador é uma boa representação da função verdadeira?
- Aplique `ajuste_exponencial` aos mesmos dados. Reporte a e b estimados, compare com os valores verdadeiros ($a = 5$, $b = -0,8$) e calcule o erro relativo de cada parâmetro.
- Plote as três curvas no mesmo gráfico: dados, interpolador e exponencial ajustada. Qual representação é mais fiel entre $x = 0$ e $x = 3$? E fora do intervalo de dados (extrapolação para $x \in [3, 4]$)?
- Discuta: se você fosse usar o modelo para prever $f(3,5)$, qual dos dois usaria? Por quê?

Q3.2 — Diagnóstico pelo gráfico de resíduos

Usando os dados do experimento anterior:

- Calcule os resíduos $r_i = y_i - \hat{y}_i$ para o ajuste exponencial. Plote r_i vs. x_i . Os resíduos têm estrutura (tendência, padrão) ou parecem aleatórios?
- Agora tente ajustar um modelo linear $y = a_0 + a_1x$ aos mesmos dados ruidosos. Plote os resíduos. O que o gráfico revela sobre a adequação desse modelo?
- Um padrão sistemático nos resíduos indica que o modelo está **subdimensionado** (*underfitting*). Resíduos sem estrutura indicam um ajuste adequado. Formule uma regra prática para diagnosticar *underfitting* a partir do gráfico de resíduos.

4. Modelos Não Lineares e Linearização

Q4.1 — Crescimento populacional e lei de potência

A tabela abaixo apresenta dados históricos da população urbana brasileira (em milhões):

Ano	1960	1970	1980	1990	2000	2010
Pop. (M)	32	52	82	111	138	161

Defina $t = \text{Ano} - 1960$ (anos desde 1960).

(a) Teste três modelos:

- Exponencial: $P(t) = a e^{bt}$, ajustado via linearização (`ajuste_exponencial`);
- Potência: $P(t) = a t^b$, ajustado via log-log (`ajuste_potencia`);
- Polinomial grau 2: ajustado via `regressao_polinomial`.

Para cada modelo, reporte os parâmetros estimados e o R^2 .

- (b) Plote os três modelos sobre os dados. Visualmente, qual apresenta melhor qualidade de ajuste no intervalo de dados?
- (c) Use cada modelo para prever a população em $t = 60$ (ano 2020). O valor real foi aproximadamente 171 M. Calcule o erro percentual de cada modelo.
- (d) **Limitação da linearização:** ao linearizar $P = a e^{bt}$, minimizamos $\sum (\ln P_i - \ln \hat{P}_i)^2$, não $\sum (P_i - \hat{P}_i)^2$. Use `ajuste_ao_linear` com função `lambda t, a, b: a*np.exp(b*t)` para ajuste direto. Os parâmetros diferem da linearização? O R^2 (calculado no espaço original) melhora?

Q4.2 — Quando a linearização distorce

Gere dados de uma lei de potência com ruído multiplicativo:

```
np.random.seed(11)
x_p = np.linspace(1, 10, 20)
y_p = 3.0 * x_p**2.5 * (1 + 0.1 * np.random.randn(20))
```

- (a) Ajuste via linearização log-log (`ajuste_potencia`). Reporte a e b e compare com os valores verdadeiros ($a = 3$, $b = 2,5$).
- (b) Ajuste via `curve_fit` (mínimos quadrados não lineares diretos, `p0=[1, 2]`). Compare os erros relativos nos parâmetros entre os dois métodos.
- (c) Plote os resíduos de ambos os métodos no espaço original. Qual abordagem apresenta resíduos menores e mais homogêneos?
- (d) Explique intuitivamente por que o ruído multiplicativo favorece o ajuste direto em relação à linearização.

5. Comparação Integrada: Interpolação vs. Ajuste

Síntese conceitual

Esta seção não fornece código novo: usa tudo que foi construído. O objetivo é desenvolver **julgamento numérico** — a capacidade de escolher o método certo para cada tipo de problema.

Q5.1 — Tabela de síntese

Com base nos experimentos realizados nas seções anteriores, preencha a tabela abaixo. No local indicado, use os dados quantitativos obtidos experimentalmente.

Aspecto	Interpolação	Ajuste (MQ)
Passagem pelos dados		
Reação ao ruído		
Número de parâmetros		
Risco de oscilação		
Extrapolação		
Escolha do modelo		
Quando preferir		
Ferramenta típica		

Q5.2 — Tomada de decisão em cenários reais

Para cada cenário abaixo, indique qual abordagem você usaria (interpolação por quê? ajuste por quê? qual tipo?) e justifique com base nos conceitos da investigação:

- Um engenheiro de controle tem a curva $v \times t$ de um motor medida com sensores de alta precisão (erro $< 0,01\%$) em 50 instantes de tempo. Ele precisa calcular a posição por integração numérica.
- Um epidemiologista tem contagens diárias de casos de uma doença em 30 dias, com ruído de subnotificação de aproximadamente 15%. Ele precisa estimar a taxa de crescimento r no modelo $C = C_0 e^{rt}$.
- Um projetista de pontes tem 8 medições de deslocamento vs. carga em uma viga, obtidas em laboratório sem erros instrumentais significativos. Ele precisa interpolar deslocamentos para cargas intermediárias.
- Um cientista de dados tem 500 pares (x_i, y_i) de medições físicas com ruído gaussiano. Ele suspeita que $y = ax^b + c$, mas não tem certeza do valor de c .

6. Projeto Integrador: Calibração de Sensor

Contexto

Um sensor de pressão industrial gera leituras em milivolts (mV) que precisam ser convertidas em pressão (bar). O fabricante fornece 8 pontos de calibração (obtidos em laboratório, sem erros), mas o técnico suspeita que as leituras em campo contêm ruído de ± 2 mV.

Dados de calibração (laboratório):

Tensão (mV)	10	20	35	50	65	80	90	100
Pressão (bar)	0,5	1,2	2,3	3,5	4,8	6,0	6,8	7,5

Q6.1 — Calibração precisa (laboratório)

- Usando os dados de calibração (sem ruído), ajuste:
 - Spline* cúbica `not-a-knot`;
 - Regressão polinomial de grau 2 e grau 3;
 - Modelo linear simples.
- Para cada método, calcule o resíduo máximo $\max_i |y_i - \hat{y}_i|$ nos 8 pontos de calibração.
- Plote todas as curvas em $[10, 100]$ mV com 200 pontos de avaliação. Visualmente, qual apresenta o comportamento mais suave?
- Se a precisão exigida for $\pm 0,05$ bar, qual(is) método(s) é(são) aceitável(is)? Justifique numericamente.

Q6.2 — Robustez ao ruído de campo

Simule 30 leituras de campo com ruído:

```
np.random.seed(42)
mV_lab = np.array([10, 20, 35, 50, 65, 80, 90, 100], dtype=float)
bar_lab = np.array([0.5, 1.2, 2.3, 3.5, 4.8, 6.0, 6.8, 7.5])
mV_campo = np.linspace(10, 100, 30)
ruído = 2.0 * np.random.randn(30) # +/- 2 mV ruído
# Use a spline de calibração para gerar "verdade" e adicione ruído:
from interpolacao import spline_cubica
p_verdade, _ = spline_cubica(mV_lab, bar_lab, mV_campo)
mV_ruidoso = mV_campo + ruído # entrada ruidosa
```

- Aplique a *spline* cúbica de calibração para converter `mV_ruidoso` em pressão. Plote pressão estimada vs. pressão verdadeira (`p_verdade`) e calcule o RMSE.

- (b) Re-ajuste um polinômio de grau 3 diretamente sobre os 30 pontos ruidosos e calcule o RMSE. Compare com o método anterior.
- (c) Qual abordagem é mais robusta: usar a spline de calibração do laboratório com entradas ruidosas, ou reajustar o modelo com os dados ruidosos? Justifique com os valores de RMSE.
- (d) O técnico propõe interpolar (Lagrange grau 29) os 30 pontos ruidosos de campo. Sem executar o código, preveja o que aconteceria com o RMSE e com as oscilações. Em seguida, execute e confirme sua previsão.

Q6.3 — Decisão final e documentação

- (a) Você é o engenheiro responsável. Escreva um parágrafo técnico (máximo 10 linhas) recomendando o método de conversão $\text{mV} \rightarrow \text{bar}$ para uso em campo. Inclua:
 - o método escolhido e sua justificativa;
 - o erro esperado;
 - as condições em que a recomendação deixa de ser válida.
- (b) Proponha uma estratégia de **re-calibração periódica**. Com que frequência e em que condições você recomendaria refazer os experimentos de laboratório? O que sinalizaria a necessidade de re-calibração antes do prazo previsto?

7. Desafio (Opcional — Pontuação Extra)

Desafio Computacional

As questões desta seção são opcionais e destinadas a alunos que desejam aprofundar a investigação além do exigido.

Q7.1 — Spline de suavização (*smoothing spline*)

A *spline* de suavização resolve o compromisso entre ajuste e suavidade via o funcional:

$$J_\lambda(s) = \sum_{i=1}^n (y_i - s(x_i))^2 + \lambda \int (s''(x))^2 dx.$$

O parâmetro λ controla o trade-off: $\lambda \rightarrow 0$ interpola, $\lambda \rightarrow \infty$ ajusta uma reta.

- (a) Use `scipy.interpolate.UnivariateSpline` com parâmetro `s` (equivalente a λ) nos dados da questão Q3.1 (dados ruidosos da exponencial). Teste $s \in \{0, 0.1, 0.5, 1.0, 5.0\}$.

- (b) Plote as cinco curvas. Identifique o valor de s que oferece melhor equilíbrio visual entre ajuste e suavidade.
- (c) Calcule o RMSE em relação à função verdadeira $5e^{-0.8x}$ para cada valor de s . O RMSE mínimo coincide com o melhor visual?
- (d) Interprete o papel de λ como **regularização**. Como esse conceito se relaciona com o sobreajuste estudado em Q2.2?

Q7.2 — Interpolação de Hermite e derivadas conhecidas

Em algumas aplicações (dinâmica de veículos, animação computacional), tanto o valor $f(x_i)$ quanto a derivada $f'(x_i)$ são conhecidos nos nós.

- (a) Implemente a interpolação de Hermite cúbica para dois nós x_0, x_1 , dados $f(x_0), f(x_1), f'(x_0), f'(x_1)$. (Use as funções de base de Hermite $H_{00}, H_{10}, H_{01}, H_{11}$.)
- (b) Para $f(x) = \sin(x)$, use os nós $x_0 = 0, x_1 = \pi/2$. Compare o erro máximo em $[0, \pi/2]$ entre:
- Hermite cúbico (usa 4 informações: 2 valores + 2 derivadas);
 - Lagrange cúbico com 4 nós igualmente espaçados;
 - *Spline* cúbica com 4 nós.
- (c) Qual método é mais preciso neste caso? O resultado faz sentido intuitivamente?

Q7.3 — Regressão com múltiplas variáveis

Um engenheiro mede o consumo de combustível C (L/100 km) de 20 veículos em função de massa m (kg) e potência P (cv):

```
np.random.seed(99)
m = 1000 + 400 * np.random.rand(20)      # massa: 1000-1400 kg
P = 80 + 120 * np.random.rand(20)        # potencia: 80-200 cv
C = 0.005*m + 0.03*P - 5 + 0.5*np.random.randn(20)
```

- (a) Monte a matriz de design $\mathbf{X}$ para o modelo $C = \beta_0 + \beta_1 m + \beta_2 P$ e resolva o sistema de equações normais $\mathbf{X}^T \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^T \mathbf{C}$ usando `numpy.linalg.solve`. Compare $\hat{\boldsymbol{\beta}}$ com os coeficientes verdadeiros.
- (b) Calcule R^2 múltiplo e o erro padrão dos resíduos. O modelo é adequado?
- (c) Adicione o termo $\beta_3 mP$ (interação). O R^2 melhora significativamente? Use o AIC para decidir se o termo de interação deve ser incluído.

Entrega e Critérios de Avaliação

O que entregar

Formato de entrega (via Moodle, antes do prazo):

1. **Relatório PDF** com todas as respostas, tabelas preenchidas, gráficos gerados e análises escritas. Gráficos devem ter título, eixos rotulados e legenda.
2. **Arquivos Python organizados:** `interpolacao.py`, `ajuste.py` e `main.py` (*script* que reproduz todos os resultados do relatório).
3. Cada arquivo deve conter uma *docstring* descrevendo o que faz.

Seção	Questões	Valor
1. Interpolação: Implementação	Q1.1–Q1.4	2,0 pts
2. Ajuste de Curvas	Q2.1–Q2.3	2,0 pts
3. Dados com Ruído	Q3.1–Q3.2	1,5 pts
4. Modelos Não Lineares	Q4.1–Q4.2	1,5 pts
5. Comparação Integrada	Q5.1–Q5.2	1,0 pts
6. Projeto Integrador	Q6.1–Q6.3	2,0 pts
Total obrigatório		10,0 pts
7. Desafio (extra)	Q7.1–Q7.3	até 1,5 pts

Critérios de qualidade:

- Código limpo, comentado e executável sem erros;
- Análises com rigor: não basta dizer “o ajuste ficou melhor” — é preciso quantificar (R^2 , RMSE, erro máximo) e relacionar com a teoria;
- Gráficos com qualidade (títulos, legendas, escala adequada);
- Conexão explícita entre resultados empíricos e a teoria das aulas: erro de interpolação, critério de convergência de Runge, teorema de existência e unicidade do interpolador, sistema de equações normais;
- Discussão genuína das diferenças entre os dois paradigmas — o ponto central desta investigação.

A. Esqueleto do *Script* Principal

Listing 3: main.py — esqueleto para o relatório

```

1  """
2  main.py
3  Plano de Investigacao: Interpolacao e Ajuste de Curvas
4  Disciplina: Calculo Numerico - UNIFAL-MG
5  """
6  import numpy as np
7  import matplotlib.pyplot as plt
8  from interpolacao import lagrange, newton, diferencas_divididas, \
9      nos_chebyshev, spline_cubica
10 from ajuste import regressao_linear, regressao_polinomial, \
11     ajuste_exponencial, ajuste_potencia, ajuste_ao_linear, \
12     residuos
13
14 np.random.seed(42)
15
16 # ---- Secao 1: Verificacao Lagrange vs. Newton ----
17 x_pts = np.array([1.0, 2.0, 3.0, 4.0])
18 y_pts = np.array([1.0, 4.0, 9.0, 16.0])
19 x_av = np.array([1.5, 2.5])
20
21 val_lag = lagrange(x_pts, y_pts, x_av)
22 val_new = newton(x_pts, y_pts, x_av)
23 print(f"Lagrange : {val_lag}")
24 print(f"Newton : {val_new}")
25 print(f"Diferenca: {np.abs(val_lag - val_new)}")
26
27 # ---- Secao 1: Runge ----
28 f_runge = lambda t: 1 / (1 + 25 * t**2)
29 t_eval = np.linspace(-1, 1, 500)
30
31 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
32 for n in [5, 9, 13, 17]:
33     x_eq = np.linspace(-1, 1, n)
34     x_ch = nos_chebyshev(-1, 1, n)
35     err_eq = np.abs(f_runge(t_eval) - lagrange(x_eq, f_runge(x_eq),
36         t_eval))
37     err_ch = np.abs(f_runge(t_eval) - lagrange(x_ch, f_runge(x_ch),
38         t_eval))
39     axes[0].semilogy(t_eval, err_eq, label=f'n={n}')
40     axes[1].semilogy(t_eval, err_ch, label=f'n={n}')
41 for ax, titulo in zip(axes, ['Nos Igualmente Espacados', 'Nos de
42     Chebyshev']):
43     ax.set(title=titulo, xlabel='x', ylabel='|erro|')
44     ax.legend(); ax.grid(True, alpha=0.3)
45 plt.tight_layout()
46 plt.savefig('runge_comparacao.pdf', dpi=150)
47
48 # ---- Secao 2: Sobreajuste ----
49 x_sin = np.linspace(0, 2*np.pi, 15)
50 y_sin = np.sin(x_sin) + 0.15 * np.random.randn(15)

```

```
47 x_fine = np.linspace(0, 2*np.pi, 500)
48
49 for grau in [1, 3, 5, 7, 10, 14]:
50     coefs, r2 = regressao_polynomial(x_sin, y_sin, grau)
51     y_hat = np.polyval(coefs[::-1], x_fine)
52     err_max = np.max(np.abs(np.sin(x_fine) - y_hat))
53     print(f"Grau {grau:2d}: R2={r2:.4f} ||f-p||_inf={err_max:.4f}")
54
55 # TODO: Secao 3 - Dados com ruido
56 # TODO: Secao 4 - Populacao brasileira
57 # TODO: Secao 5 - Tabela comparativa
58 # TODO: Secao 6 - Projeto sensor de pressao
59
60 plt.tight_layout()
61 plt.savefig('resultados_interpolacao_ajuste.pdf', dpi=150)
62 plt.show()
```

B. Referências Bibliográficas

1. Burden, R. L.; Faires, J. D. *Numerical Analysis*. 10.^a ed. Cengage, 2015. Caps. 3 e 8.
2. Ruggiero, M. A. G.; Lopes, V. L. R. *Cálculo Numérico: Aspectos Teóricos e Computacionais*. 2.^a ed. Pearson, 1996.
3. Press, W. H. et al. *Numerical Recipes in C*. 3.^a ed. Cambridge, 2007. Caps. 3 e 15.
4. NumPy — <https://numpy.org/doc/stable/>
5. SciPy Interpolate — <https://docs.scipy.org/doc/scipy/reference/interpolate.html>
6. SciPy Optimize — <https://docs.scipy.org/doc/scipy/reference/optimize.html>